WEBAPP FOR RATHNAM MEN'S WORLD

Project Report Submitted to

AYYA NADAR JANAKI AMMAL COLLEGE, SIVAKASI.

affiliated to the **Madurai Kamaraj University, Madurai**,
in partial fulfilment of the requirements for the award of
the Degree of

BACHELOR OF COMPUTER APPLICATIONS

Submitted by

K. VIGNESH

Register No: 22UL46

N. RANJITH KUMAR

Register No: 22UL50

UNDER GRADUATE DEPARTMENT OF COMPUTER APPLICATIONS

AYYA NADAR JANAKI AMMAL COLLEGE

(Autonomous, affiliated to Madurai Kamaraj University, Re-accredited (4th cycle) with 'A+' Grade (CGPA of 3.48 out of 4) by NAAC, Recognized as College of Excellence and Mentor Institution by UGC, STAR College by DBT and Ranked 63rd at National Level in NIRF 2024 and DST-FIST (2024) supported & ISO 9001:2015 Certified Institution)

SIVAKASI – 626 124

APRIL – 2025

Dr. R. VENGATESHKUMAR, M.C.A., M.Phil., Ph.D.
Head,
Under Graduate Department of Computer Applications,
Ayya Nadar Janaki Ammal College (Autonomous),
Sivakasi.

CERTIFICATE

This is to certify that this project report entitled **"WEBAPP FOR RATHNAM MEN'S WORLD"** being submitted by **K. Vignesh (22UL46) and N. Ranjith Kumar (22UL50)**, final year students of Under Graduate Department of Computer Applications, Ayya Nadar Janaki Ammal College (Autonomous), Sivakasi, affiliated to Madurai Kamaraj University, Madurai, is a *bonafide* record of work carried out by them under the guidance of **Mrs. G. Rajalakshmi, M.C.A., M.E.,** Assistant Professor, Under Graduate Department of Computer Applications, Ayya Nadar Janaki Ammal College (Autonomous), Sivakasi.

Place: Sivakasi, **Signature of the Head**

Date: (**Dr. R. VENGATESHKUMAR)**

Mrs. G. RAJALAKSHMI, M.C.A., M.E.,

Assistant Professor,

Under Graduate Department of Computer Applications,

Ayya Nadar Janaki Ammal College (Autonomous),

Sivakasi.

CERTIFICATE

This is to certify that this Project report entitled **"WEBAPP FOR RATHNAM MEN'S WORLD"** being submitted by **K. Vignesh (22UL46) , N. Ranjith Kumar (22UL50)**, final year students of Under Graduate Department of Computer Applications, Ayya Nadar Janaki Ammal College (Autonomous), Sivakasi, affiliated to Madurai Kamaraj University, Madurai, is a *bonafide* record of work carried out by them under my guidance and supervision.

It is further certified that to the best of my knowledge, this project report or any part thereof has not been submitted in this College or elsewhere for the award of any other degree or diploma.

Place: Sivakasi, **Signature of the Guide**

Date: **(Mrs. G. Rajalakshmi)**

CERTIFICATE

This is to certify that we have examined the report entitled **"WEBAPP FOR RATHNAM MEN'S WORLD"** being submitted by **K. Vignesh (22UL46), N. Ranjith Kumar (22UL50) ,**final year students of Under Graduate Department of Computer Applications, Ayya Nadar Janaki Ammal College (Autonomous), Sivakasi, affiliated to Madurai Kamaraj University, Madurai, is a *bonafide* record of work carried out by them under the guidance of **Mrs. G. Rajalakshmi, M.C.A.,M.E.,** Assistant Professor, Under Graduate Department of Computer Applications, Ayya Nadar Janaki Ammal College (Autonomous), Sivakasi, and have evaluated their performance at *viva-voce* conducted on / / 2025.

Internal Examiner **External Examiner**

K. VIGNESH (22UL46),

N. RANJITH KUMAR (22UL50),

III BCA,

Under Graduate Department of Computer Applications,

Ayya Nadar Janaki Ammal College (Autonomous),

Sivakasi.

DECLARATION

This project report entitled "**WEBAPP FOR RATHNAM MEN'S WORLD**", has been carried out by us at the Under Graduate Department of Computer Applications, Ayya Nadar Janaki Ammal College (Autonomous), Sivakasi, affiliated to Madurai Kamaraj University, Madurai, in partial fulfillment of the requirements for the award of the Degree of Bachelor of Computer Applications.

We further declare that this project report or any part thereof has not been submitted in this College or elsewhere for the award of any other degree or diploma.

Place: Sivakasi, **Signature of the Candidates**

Date: **K. VIGNESH**

N. RANJITH KUMAR

ACKNOWLEDGEMENT

At the outset we express gratitude to **God** almighty for the opportunity to work on this project and for the strength and guidance to complete it successfully.

We heartily express our sincere thanks to our **College Management** and our beloved Principal **Dr. C. ASHOK, M.P.Ed., M.Phil., D.Y.Ed., Ph.D.** for providing us this great opportunity and complete facility during this project.

We heartily express our sincere thanks to **Dr. R. VENGATESHKUMAR, M.C.A., M.Phil., Ph.D.** Head, Under Graduate Department of Computer Applications for his valuable advice and guidance throughout this Project work.

We heartily express our sincere thanks to **Mrs. G. RAJALAKSHMI, M.C.A., M.E.,** Assistant Professor, Department of Computer Applications for the excellence guidance, encouragement and personal commitments manifested throughout this Project work.

We extend our sincere thanks to all our **department staff members** for their excellent support during this Project work.

We also thank **our parents and friends** for their encouragement and affection which helped us much to involve enthusiastically in our Project work.

K. VIGNESH

N. RANJITH KUMAR

CHAPTER NO	CONTENTS	PAGE NO
1.	**Synopsis**	**1**
2.	**System Requirements**	**2**
	2.1 Hardware Requirements	2
	2.2 Software Requirements	2
3.	**System Description**	**3**
	3.1 Windows 11	3
	3.2 React js	6
	3.3 Node js and Express js	8
	3.4 Mongo db	10
4.	**System Design**	**12**
	4.1 Data Flow Diagram	12
	4.2 Database Design	17
5.	**System Implementation**	21
	5.1 Modules Description	21
	5.2 Screenshots	24
6.	**System Testing**	**33**
	6.1 Unit Testing	33
	6.2 Integration Testing	35
7.	**System Maintenance**	**38**
8.	**Conclusion**	**40**
APPENDIX - A	**BIBLIOGRAPHY**	**A1**

1. SYNOPSIS

The system entitled as **Webapp for Rathnam Men's World** is a dedicated online platform offering stylish and trendy clothing for men. This web app provides a smooth shopping experience, featuring a wide range of men's fashion items in various sizes. Users can explore and purchase the latest collections of shirts, pants, and t-shirts, all designed to keep up with modern fashion trends. With secure online payment options and a user-friendly interface, shopping is quick, safe, and convenient. The platform ensures a smooth checkout process, making it easier for customers to complete their purchases without hassle. Additionally, users can track their orders and receive timely updates on their purchases. The stock is efficiently managed and updated by the admin to ensure the availability of the latest products. High-quality images and detailed product descriptions help customers make informed decisions. Rathnam Men's World is committed to delivering premium-quality fashion at affordable prices while maintaining excellent customer satisfaction.

2. SYSTEM REQUIREMENTS

The system requirements includes the hardware and software requirements that helps to run the application. System requirements outline the hardware and software specification necessary for a particular software application or system to operate effectively. These requirements ensure that users have the necessary infrastructure in place to run the software smoothly and without issue. By understanding and meeting the system requirements outlined for a software application, users can ensure that they have the necessary hardware and software infrastructure in place to run the software effectively and achieve optimal performance.

2.1 HARDWARE REQUIREMENTS

Processor	: 11th Gen Intel(R) Core(TM) i3-1215U
RAM	: 8.00 GB
Hard disk	: 1 TB

2.2 SOFTWARE REQUIREMENTS

OPERATING SYSTEM	:	Windows 11
FRONT END	:	React js
BACK END	:	Node js, Express js, Mongo db

3. SYTEM DESCRIPTION

WINDOWS 11

Windows 11 is the latest major release of Microsoft's Windows NT operating system. It is the successor to Windows 10, which had been released five years earlier. Windows 11 was officially announced on June 24, 2021, and became generally available on October 5, 2021. The operating system introduces a refreshed design and several new features aimed at improving user experience, performance, and security. It is available as a free upgrade for eligible Windows 10 devices via Windows Update.

One of the most notable changes in Windows 11 is its redesigned user interface, featuring a centered Start menu and taskbar, rounded corners, and revamped system icons, contributing to a more modern and streamlined look. The operating system also introduces features like Snap Layouts and Snap Groups, which improve multitasking and window management, and virtual desktops have been enhanced with more customization options.

In terms of hardware requirements, Windows 11 has stricter minimum system requirements than its predecessor. It requires a 64-bit processor with at least 1 GHz clock speed and two or more cores, 4 GB of RAM, and 64 GB of storage. Additionally, Windows 11 mandates the presence of TPM (Trusted Platform Module) 2.0, a secure boot process with UEFI firmware, and a DirectX 12-compatible graphics card for the best experience. The introduction of these requirements has meant that some older devices are not eligible for an upgrade to Windows 11.

Windows 11 places a stronger emphasis on integration with Microsoft’s cloud services, including OneDrive, Microsoft 365, and Azure Active Directory, enabling enhanced productivity and collaboration. Furthermore, Windows 11 brings a new Microsoft Store that hosts a wider range of apps, including Android apps through the Amazon Appstore, offering users even more flexibility.

Windows 11 also brings performance improvements, with better memory management, faster wake-from-sleep times, and increased battery efficiency for portable devices. Additionally, Windows 11 integrates Xbox Game Pass and other gaming features, enhancing the gaming experience for users, while also introducing support for Auto HDR and DirectStorage, promising faster load times and higher-quality visuals.

As of early 2022, Windows 11 has gradually gained traction, with adoption rates expected to rise as more devices meet the hardware requirements. However, Windows 10 will remain

supported until October 14, 2025, allowing businesses and consumers more time to transition to Windows 11. Windows 11 is poised to continue evolving with regular updates, providing new features and security enhancements for the long term

Development

At the Microsoft Worldwide Partner Conference in 2011, Andrew Lees, the chief of Microsoft's mobile technologies, said that the company intended to have a single software ecosystem for PCs, phones, tablets, and other devices. In December 2013, technology writer Mary Jo Foley reported that Microsoft was working on an update to Windows 8 codenamed "Threshold", after a planet in its Halo franchise. Similarly to "Blue" (which became Windows 8.1) Foley described Threshold, not as a single operating system, but as a "wave of operating systems" across multiple Microsoft platforms and services, quoting Microsoft sources, scheduled for the second quarter of 2015. User also stated that one of the goals for Threshold was to create a unified application platform and development toolkit for Windows, Windows Phone and Xbox One.

At the Build Conference in April 2014, Microsoft's Terry Myerson unveiled an updated version of Windows 8.1 (build 9697) that added the ability to run Windows Store apps inside desktop windows and a more traditional Start menu in place of the Start screen seen in Windows 8. The new Start menu takes after Windows 7's design by using only a portion of the screen and including a Windows 7-style application listing in the first column.

The second column displays Windows 8-style app tiles. Myerson said that these changes would occur in a future update, but did not elaborate. Microsoft also unveiled the concept of a "universal Windows app", allowing Windows Store apps created for Windows 8.1 to be ported to Windows Phone 8.1 and Xbox One while sharing a common codebase, with an interface designed for different device form factors, and allowing user data and licenses for an app to be shared between multiple platforms. Windows Phone 8.1 would share nearly 90% of the common Windows Runtime APIs with Windows 8.1 on Pc

Features

Windows 11 introduces a new evolution of universal apps, expanding on the concepts first seen in Windows 8 and refined in Windows 10. These apps, now part of the Universal Windows Platform (UWP), are designed to run seamlessly across a wide range of devices, including PCs, tablets, smartphones, Xbox Series X/S, Surface Hub, and Mixed Reality devices, all while maintaining consistency in code and user experience. Windows 11 continues to bridge the gap between mouse-oriented and touchscreen-optimized interfaces, especially on 2-in-1 devices, by offering an adaptive user interface that adjusts depending on the available input methods.

The Start menu in Windows 11 has undergone further refinement, offering a centered design with a more streamlined, minimalistic approach while keeping elements that reflect Windows' legacy. Additionally, Windows 11 brings several new features, including the revamped **Microsoft Edge** browser, enhanced virtual desktop capabilities, **Task View** for improved window and desktop management, and greater integration with Microsoft's cloud-based services.

One of the most notable changes in Windows 11 is its ability to better integrate with both ARM-based and x86-based systems. ARM devices running Windows 11 can use x86 applications through 32-bit emulation, with enhanced performance for these applications. **Windows Store** in Windows 11 continues to serve as a centralized marketplace for apps, games, movies, and other digital content. Microsoft is also expanding the variety of apps available on the Store, now offering support for Android apps, which run on Windows 11 through the Amazon Appstore, increasing app diversity and usability.

Windows 11 builds on Windows 10's universal app ecosystem by improving and optimizing the Universal Windows Platform (UWP), enabling developers to create applications that are compatible across devices such as PCs, tablets, smartphones, Xbox, and even Surface Hub. To help developers, Microsoft provides a rich set of development tools, frameworks, and resources to support app creation, testing, and distribution, making Windows 11 a more developer-friendly platform.

Windows 11 also includes improvements in security and performance, such as a new TPM 2.0 (Trusted Platform Module) requirement and additional hardware-based

security features. These enhancements offer better protection against modern threats while maintaining the performance improvements seen in previous iterations.

3.2 REACT

React is a popular JavaScript library for building user interfaces, primarily for single-page applications (SPAs) where user need a dynamic, interactive user experience. It is maintained by Facebook and a community of developers. React enables the creation of reusable UI components, making the development process more efficient and modular. Unlike traditional HTML, React uses a declarative approach to building web interfaces, allowing developers to describe how the UI should look based on the current state of the application.
React allows developers to build complex UIs from simple components, which are small, self-contained pieces of code that manage their state and can be reused throughout the application. Components can be either functional or class-based, though the trend is now shifting towards functional components due to React's Hooks API, which simplifies managing state and side effects.

React works with a virtual DOM (Document Object Model) that optimizes performance by minimizing direct manipulation of the actual DOM. When a change occurs in the state of the app, React updates the virtual DOM first, compares it with the previous version, and only updates the real DOM with the differences, leading to faster rendering.

While React doesn't provide built-in features for things like routing or state management, it can be integrated with other libraries such as React Router for navigation and Redux for state management. Additionally, React can be paired with CSS for styling, or CSS-in-JS libraries like styled-components for a more dynamic approach to styling.

React can be used in conjunction with other technologies like Node.js for backend development, allowing for the full stack development of web applications. One of React's key strengths is its ability to efficiently render large datasets and provide a smooth, interactive user experience.

In recent years, React has become the go-to tool for developing complex, responsive web applications, powering sites and apps such as Facebook, Instagram, and Netflix. It continues to evolve with frequent updates and an active developer community.

TAILWIND CSS

Tailwind CSS is a utility-first CSS framework designed to make web development faster and more efficient by providing low-level utility classes that can be composed to build custom designs. Unlike traditional CSS frameworks that come with predefined components, Tailwind CSS encourages a more flexible and customizable approach to styling by using utility classes directly in the HTML markup. This results in a more streamlined development process, as developers can create responsive and custom designs without writing custom CSS for every element.

Tailwind's utility classes cover a wide range of styling options, including layout, spacing, typography, colors, borders, shadows, and more. This approach enables developers to rapidly prototype designs and tweak the styles directly in the HTML without having to write separate CSS rules. One of the key benefits of using Tailwind CSS is its focus on reusability and modularity, as each utility class represents a single, reusable styling property, making it easy to maintain and scale a project.

Tailwind also includes an advanced configuration system, allowing developers to customize their design system by adjusting values like colors, fonts, breakpoints, and spacing to suit their specific needs. With Tailwind's built-in support for responsiveness, developers can design mobile-first layouts by adding responsive classes that apply styles at different breakpoints.

In addition, Tailwind CSS integrates well with modern JavaScript frameworks like React, Vue, and Angular, offering an efficient way to build dynamic user interfaces while maintaining design consistency. Developers can also use Tailwind CSS with PostCSS or PurgeCSS to optimize the final output by removing unused CSS classes, ensuring minimal file sizes and better performance.

Tailwind CSS is not just about utility classes; it also supports custom components and utility generation, meaning developers can build reusable component libraries without writing custom CSS. This makes Tailwind CSS a powerful choice for projects ranging from small websites to large-scale applications, offering both flexibility and scalability.

History

Tailwind CSS was created by Adam Wathan in 2017 as an alternative to traditional CSS frameworks like Bootstrap and Foundation. Its unique utility-first approach gained popularity quickly within the developer community for its ability to streamline workflows and

simplify the design process. Over time, Tailwind has evolved with features like JIT (Just-in-Time) mode for on-demand class generation, offering even more flexibility and efficiency.

3.3 EXPRESS JS

Express is a fast, minimalist web application framework for Node.js, designed to simplify the process of building robust and scalable server-side applications. Originally released in 2010, Express quickly became the most popular framework for Node.js, providing a powerful set of features for building web and mobile applications. It is lightweight, flexible, and unopinionated, meaning developers have the freedom to structure their applications the way they want, without being forced into specific patterns or conventions.

Express provides a straightforward interface for handling HTTP requests and responses, and it supports routing, middleware, and template engines for rendering dynamic content. The framework enables easy management of URLs, request handling, and response generation, making it ideal for building RESTful APIs, single-page applications (SPAs), and even complex full-stack applications.

Express also allows for easy integration with databases, file systems, and external APIs, allowing developers to build applications that handle everything from user authentication to real-time communication. Middleware functions in Express provide powerful tools for adding features such as logging, error handling, authentication, and session management.

Due to its simplicity and efficiency, Express has become a core component of the MEAN and MERN stack (MongoDB, Express, Angular/React, Node.js), widely adopted by developers for building scalable and high-performance web applications. Express is open-source, supported by a large community of contributors, and is frequently updated to improve security, performance, and usability.

Express is a foundational tool for Node.js developers, allowing them to rapidly develop server-side applications while maintaining flexibility and control. It is known for its fast learning curve and its ability to handle high-traffic applications, making it one of the most trusted web frameworks in the development community.

History

Express was created by TJ Holowaychuk in 2010 as part of his broader effort to make building web applications with Node.js simpler. It was initially conceived as a minimal framework to streamline the development of HTTP servers, but it soon grew to become the foundation for many modern web applications. Over the years, Express has gained popularity

due to its ease of use and the extensive ecosystem of plugins and tools that extend its functionality.

NODE JS

Node.js is a powerful, open-source, cross-platform runtime environment that allows developers to run JavaScript on the server side. Originally developed by Ryan Dahl in 2009, Node.js uses the V8 JavaScript engine (the same engine that powers Google Chrome) to execute JavaScript code outside of the browser. This makes it possible for developers to use JavaScript for both client-side and server-side programming, creating a more streamlined development experience.

Node.js is particularly well-suited for building scalable, high-performance applications that need to handle a large number of concurrent connections. Unlike traditional server-side platforms, which use multi-threading to handle requests, Node.js operates on a single-threaded, event-driven, non-blocking I/O model. This means it can process multiple requests asynchronously, without waiting for one request to finish before moving to the next, making it ideal for I/O-heavy applications such as web servers, real-time applications, and APIs.

One of Node.js's greatest strengths is its vast ecosystem of libraries and packages available through **npm** (Node Package Manager), the largest package manager in the world. This allows developers to quickly add pre-built functionality to their applications, such as authentication, database connectivity, and file handling, saving significant development time.

Node.js is commonly used for building web servers, APIs, microservices, real-time applications (such as chat applications, gaming servers, and collaboration tools), and even command-line tools. Its ability to scale easily and handle numerous simultaneous connections makes it a popular choice for high-traffic web applications.

Because it uses JavaScript, Node.js is favored by full-stack JavaScript developers, who can work across the entire stack using the same language. Furthermore, it integrates well with popular frontend frameworks like React, Angular, and Vue.js, making it a go-to solution for building modern, dynamic, and responsive web applications.

History

Node.js was created in 2009 by Ryan Dahl to address the limitations of traditional server-side languages that rely on blocking I/O operations. Dahl's goal was to create a framework that could handle a large number of simultaneous connections in a non-blocking manner. Over time, Node.js gained a strong developer community, and its popularity grew,

especially as its asynchronous, event-driven model suited the needs of modern web development. In 2011, **npm** (Node Package Manager) was introduced, which greatly enhanced the Node.js ecosystem by providing an easy way to share and manage code libraries. Today, Node.js continues to evolve and is supported by a large open-source community.

Node.js is a powerful, open-source, cross-platform runtime environment that allows developers to execute JavaScript code server-side. One of its standout features is its non-blocking, event-driven architecture, which enables high concurrency and makes it ideal for building scalable, real-time applications like chat apps or live-streaming services. Node.js uses a single-threaded event loop, allowing it to handle multiple requests simultaneously with minimal overhead. It also boasts a vast ecosystem of packages available through the Node Package Manager (NPM), making it easy to integrate third-party libraries and tools. Its asynchronous nature ensures fast execution of I/O operations, while its use of JavaScript allows full-stack development with a single language. Additionally, Node.js is highly efficient for building microservices, APIs, and web servers due to its lightweight design and fast performance.

3.4 MONGODB

MongoDB is a widely used, open-source, NoSQL database that stores data in a flexible, JSON-like format called BSON (Binary JSON). It was designed to handle large volumes of unstructured or semi-structured data with ease, making it well-suited for modern, data-driven applications. Unlike traditional relational databases that store data in tables with predefined schemas, MongoDB allows for dynamic and flexible data models, making it particularly useful in projects where the data structure can evolve over time or doesn't fit neatly into rows and columns.

One of MongoDB’s core features is its ability to scale horizontally across multiple servers, making it a powerful solution for applications that require high availability and fast data access. MongoDB achieves this by using sharding, where data is distributed across multiple machines, ensuring that large datasets are evenly partitioned for better performance and fault tolerance.

MongoDB stores data as documents within collections, which are analogous to rows in a relational database. Each document is a JSON-like object, and collections hold multiple documents, which can have different fields and data types. This flexible schema allows developers to store varied data formats in the same collection, reducing the need for complex joins and providing easier management of complex relationships.

The database also provides powerful querying capabilities, including support for rich, multi-field queries, full-text search, and aggregation operations. MongoDB's aggregation framework allows for complex data transformations, calculations, and groupings, making it an ideal choice for applications with complex reporting needs.

MongoDB integrates seamlessly with modern web technologies, such as Node.js, and is often used in conjunction with frameworks like Express in the popular MEAN (MongoDB, Express, Angular, Node.js) or MERN (MongoDB, Express, React, Node.js) stacks. Its flexibility, scalability, and performance make MongoDB a go-to database for building data-intensive applications like content management systems, real-time analytics, and IoT platforms.

History

MongoDB was created by Dwight Merriman, Eliot Horowitz, and Kevin Ryan in 2007, with the aim of providing a database solution that could handle large amounts of unstructured data while offering flexibility and scalability. The founders were frustrated with the limitations of traditional relational databases, particularly in handling large-scale web applications. MongoDB was first released in 2009, and it quickly gained traction for its ease of use, flexibility, and ability to scale. Over the years, MongoDB has become one of the most popular NoSQL databases, widely adopted by developers across various industries and supported by a vibrant open-source community. In 2013, MongoDB, Inc. was founded to provide professional support and services, making MongoDB a mainstream database choice for enterprises as well as start

MongoDB is a popular NoSQL database known for its flexibility and scalability. It stores data in a JSON-like format called BSON (Binary JSON), which allows for the storage of complex, hierarchical data structures. One of its standout features is its ability to scale horizontally across multiple servers, which helps to handle large volumes of data and high traffic. MongoDB supports a rich query language, including filtering, aggregation, and indexing, allowing for efficient data retrieval. It also offers features like automatic sharding for distributing data, built-in replication for high availability, and strong consistency through its default write concern. Additionally, MongoDB's schema-less design enables rapid development and iteration, making it a popular choice for modern, dynamic applications.

4. SYSTEM DESIGN

4.1 DATA FLOW DIAGRAM

A Data Flow Diagram (DFD) is a graphical representation of the “flow” of data through an information system, modeling its process aspects. Often they are preliminary step used to create an overview of the system which can later be elaborated. DFDs can also be used for the visualization of data processing. External entities represent sources or destinations of data outside the system being modeled. They interact with the system by providing input data or receiving output data. External entities are typically depicted as squares or rectangles. Processes represent the actions or transformations that occur within the system. They describe how data is processed, manipulated, or transformed from inputs to outputs. Processes are depicted as circles or ovals.

Context Diagram

A context diagram is a top level (also known as "Level 0") data flow diagram. It only contains one process node ("Process 0") that generalizes the function of the entire system in relationship to external entities.

DFD Layers

Draw data flow diagrams can be made in several nested layers. A single process node on a high level diagram can be expanded to show a more detailed data flow diagram. Draw the context diagram first, followed by various layers of data flow diagrams.

DFD Levels

The first level DFD shows the main processes within the system. Each of these processes can be broken into further processes until you reach pseudo code.

A data flow diagram (DFD) is a visual representation of the information flow through a process or system. DFDs help you better understand process or system operations to discover potential problems, improve efficiency, and develop better processes. They range from simple overviews to complex, granular displays of a process or system. DFDs are visual representations that can help almost anyone grasp a system‘s or process’ logic and functions. Aside from being accessible, they provide much-needed clarity and improve productivity. A set of parallel lines shows a place for the collection of data items. A data

store indicates that the data is stored which can be used at a later stage or by the other processes in a different order. The data store can have an element or group of elements.

A DFD shows what kinds of information will be input to and output from the system, where the data will come from and go to and where the data will be stored. It does not show information about the timing of processes or information about whether processes will operate in sequence or in parallel.

DATA FLOW DIAGRAM SYMBOLS

Entities

Entities include source and destination of the data. Entities are represented by rectangle with their corresponding names.

Figure: 5.1.1 Entity

Process

The tasks performed on the data is known as process. Process is represented by circle.

Somewhere round edge rectangles are also used to represent process.

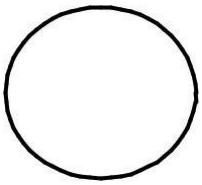


Figure: 5.1.2 Process

Data Storage

Data storage includes the database of the system. It is represented by rectangle with both smaller sides missing or in other words within two parallel lines.

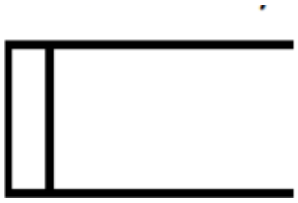


Figure: 5.1.3 Data Store

Data Flow

The movement of data in the system is known as data flow. It is represented wth the help of arrow. The tail of the arrow is source and the head of the arrow is destination.

Figure: 5.1.4 Data Flow

Level 0

Figure: 5.1.5 Context level Diagram

Level 1.1

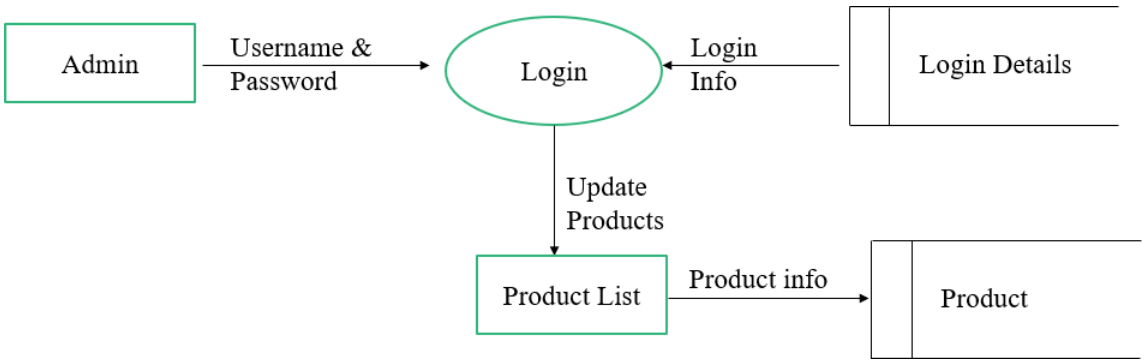


Figure: 5.1.6 Admin managing products

Level 1.2

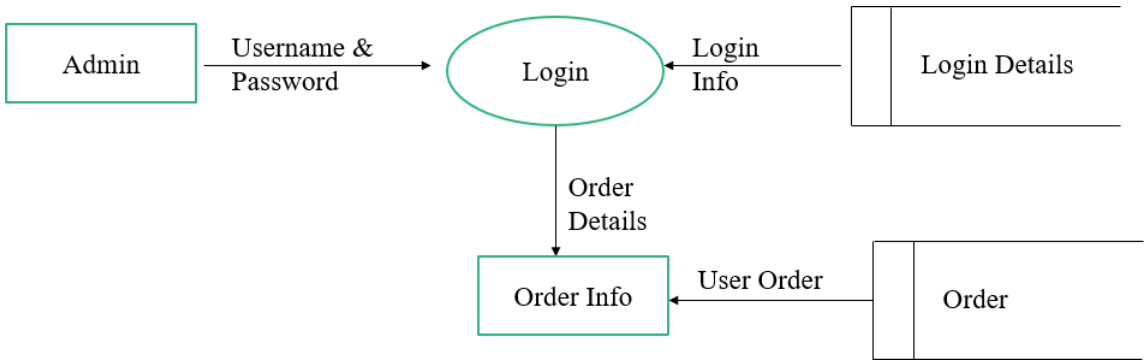


Figure: 5.1.7 Admin handling orders

Level 2.1

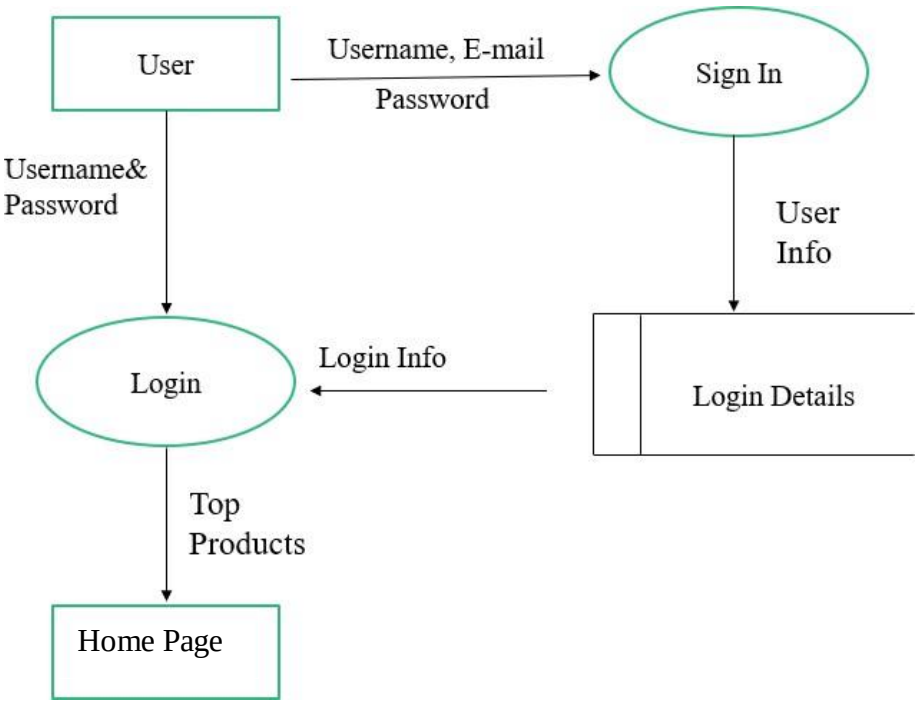


Figure: 5.1.8 User Login

Level 2.2:

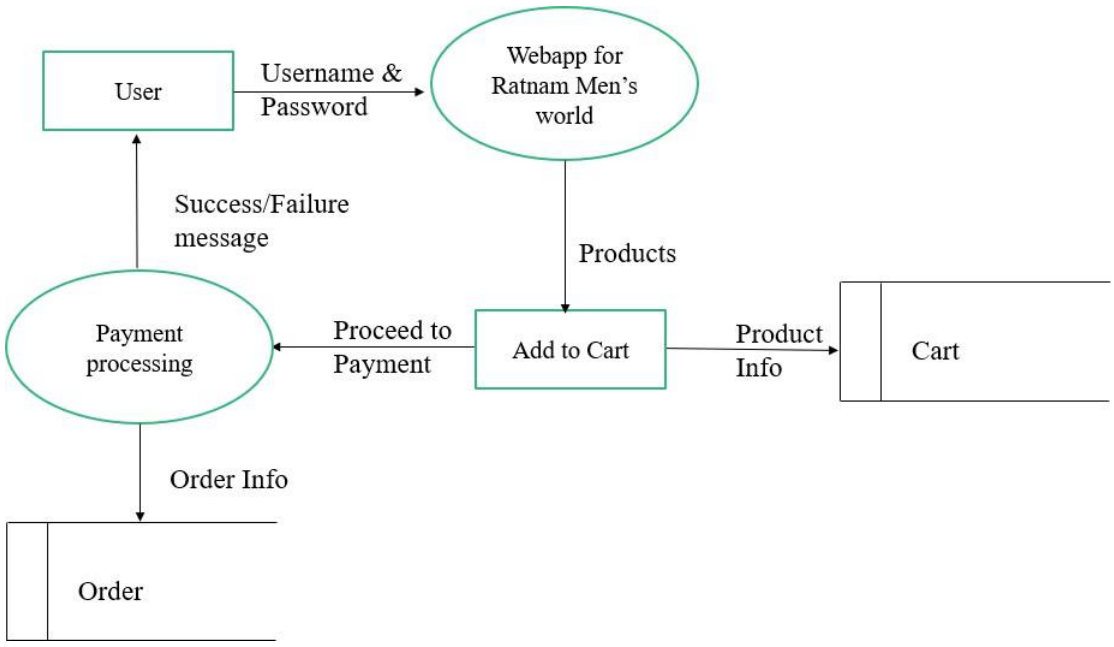


Figure: 5.1.9 User ordering products

4.2 DATABASE DESIGN

A database is a collection of interrelated data stored with minimum redundancy to serve users more quickly and efficiently. The general objective of a database is to make information access easy, quick, inexpensive, integrated and shared by different applications and users.

Database design is an important yet sometimes overlooked part of the application development lifecycle. The concepts and techniques used when designing an effective database includes:

An entity is a logical collection of things that are relevant to r database. The physical counter part of an entity is a database table. An attribute is a descriptive or quantitative characteristic of an entity. The physical counter part of an attribute is a database column (or field).

Good database design focuses on normalization, which helps eliminate redundancy and ensures data integrity by organizing it into separate, logically related tables. It also involves indexing to speed up queries, defining primary and foreign keys to establish relationships, and ensuring scalability for future growth. A well-designed database should meet the needs of the application, handle high volumes of data efficiently, and maintain flexibility for future changes, all while minimizing the risk of anomalies and inconsistencies. Proper database design is crucial for optimizing performance, ensuring data accuracy, and supporting the long-term success of an application.

Table: 4.2.1 Sign up

Column name	Data Type	Constraints	Description
_id	ObjectId	Primary Key, Auto-generated	Unique user ID (MongoDB default)
name	String	Required, Min: 3, Max: 50	User's full name
email	String	Required, Unique, Valid Email	User's email ID
password	String	Required, Min: 6	Hashed user password
phone	String	Optional, Unique, Length: 10	User's phone number
address	String	Optional	User's address
createdAt	Date	Default: Date.now	Timestamp when user registered
updatedAt	Date	Auto-updated on changes	Timestamp when user info is updated

Table: 4.2.2 Login

Column Name	Data Type	Constraints	Description
login_id	VARCHAR(50)	PRIMARY KEY	Unique ID for each login entry
user_id	VARCHAR(50)	FOREIGN KEY (users)	Links login to a user
email	VARCHAR(100)	UNIQUE NOT NULL	User email for login
password_hash	VARCHAR(255)	NOT NULL	Hashed password for security
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Account creation time
updated_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP	Last update time

Table: 4.2.3 Product Info

Column Name	Data Type	Constraints	Description
_id	ObjectId	Primary Key, Auto-generated	Unique product ID (MongoDB default)
name	String	Required, Unique, Min: 3, Max: 100	Product name
description	String	Required, Min: 10	Brief product description
price	Number	Required, Min: 0	Product price
category	String	Required, Enum: ["Shirt", "Pant", "T-shirt", "Accessories", etc.]	Product category
stock	Number	Required, Min: 0, Default: 0	Available stock quantity
images	Array[String]	Required	Array of image URLs
createdAt	Date	Default: Date.now	Timestamp when product was added
updatedAt	Date	Auto-updated on changes	Timestamp when product info is updated

Table: 4.2.4 Cart

Column Name	Data Type	Constraints	Description
_id	ObjectId	Primary Key, Auto-generated	Unique cart ID (MongoDB default)
userId	ObjectId	Required, References User	ID of the user who owns the cart
items	Array	Required	List of items added to the cart
items[].productId	ObjectId	Required, References Product	ID of the product added to cart
items[].quantity	Number	Required, Min: 1, Default: 1	Number of units of the product
items[].price	Number	Required, Min: 0	Price of the product at the time of adding to cart
createdAt	Date	Default: Date.now	Timestamp when cart was created
updatedAt	Date	Auto-updated on changes	Timestamp when cart is updated

Table: 4.2.5 Order Info

Column Name	Data Type	Constraints	Description
order_id	VARCHAR(50)	PRIMARY KEY	Unique ID for each order
user_id	VARCHAR(50)	FOREIGN KEY (users)	Links order to a user
total_price	DECIMAL(10,2)	NOT NULL	Total amount for the order
payment_status	ENUM('Pending', 'Completed', 'Failed')	DEFAULT 'Pending'	Tracks payment state
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Order placed time
updated_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP	Last update time

Table: 4.2.6 Payment Info

Column Name	Data Type	Constraints	Description
payment_id	VARCHAR(50)	PRIMARY KEY	Unique ID for each payment
order_id	VARCHAR(50)	FOREIGN KEY (orders)	Links payment to an order
user_id	VARCHAR(50)	FOREIGN KEY (users)	Links payment to a user
amount	DECIMAL(10,2)	NOT NULL	Total payment amount
transaction_id	VARCHAR(100)	UNIQUE	Transaction ID from payment gateway
payment_date	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Date and time of payment

5. SYSTEM IMPLEMENTATION

5.1 MODULE DESCRIPTION

The project is entitled as **"Webapp for Rathnam Men's World"** developed using React js as frontend and Node js, Express js and Mongo Db as back end. The project has the following modules.

- Administrator Module
- User Module

Administrator Module

In this Module, the administrator can view this web application and submit client details, men's clothing and order details etc.. The administrator module contains the following sub modules such as

- Login module
- Order Info
- User Info
- Add Products

Login module

This module allows admin to enter their user name and password to login to the admin dashboard

Add Products

In this module the admin can view the product info and the admin can also can add, delete and update outfits like shirt, t-shirt and pants for men with the detailed description for each outfit.

Order Info

This module allows the administrator to access user order information, including the outfit, quantity, and total price.

User Info

This module contains the basic information about the users who are already registered in the webapp. The admin can view how many users registered in the web app. If the new user enters or the user detail is changed, it automatically updates with the new data.

User Module

In this module, users can browse outfits, including men's clothing items like shirts, t-shirts, and pants. The user can register in the webapp by entering their username, mail, and password. They can add items to the cart, proceed to checkout, and place orders. Payments can be made securely using Razorpay.

The user module contains sub modules such as

- Login Module
- Register Module
- Home Module
- Products Module
- Cart Module
- Payment Module

Login Module

Login Module allows users to log in by entering their username and password. It works with authentication to verify whether user is already present or not.

Register Module

The register module allows users to create an account by entering their name, email, and password. It ensures security of the password by hashing. This module helps with user authentication and access control, making it easy for users to log in smoothly. Additionally, it includes input validation to prevent invalid or duplicate registrations.

Home Module

The Home Module serves as the main landing page, providing users with an overview of the mens's clothing. It showcases featured outfits, latest updates and essential navigation links. This module enhances user engagement by offering a visually appealing and user-friendly interface.

Products Module

In the product module, the user can select their outfit based on their sizes, like S, M, L, and XL. The user can search for outfits by category, if the user searches for shirt, the shirt items will appear first. The user can view detailed descriptions of the outfits, including their brand and fabric material, and they can select their desired quantities.

Cart Module

In the cart module the user can see their outfits information like quantity and their price. It calculates the total price automatically. The user can check whether the selected products are present in the cart and the user can also remove from it.

Payment Module

The Payment Module in Rathnam Men's World ensures a secure and seamless transaction process for customers. It supports multiple payment methods, including credit/debit cards, UPI, net banking, and wallets, through Razorpay integrations. To enhance security, the module uses encryption and tokenization to protect sensitive payment details.

5.1 SCREEN SHOTS

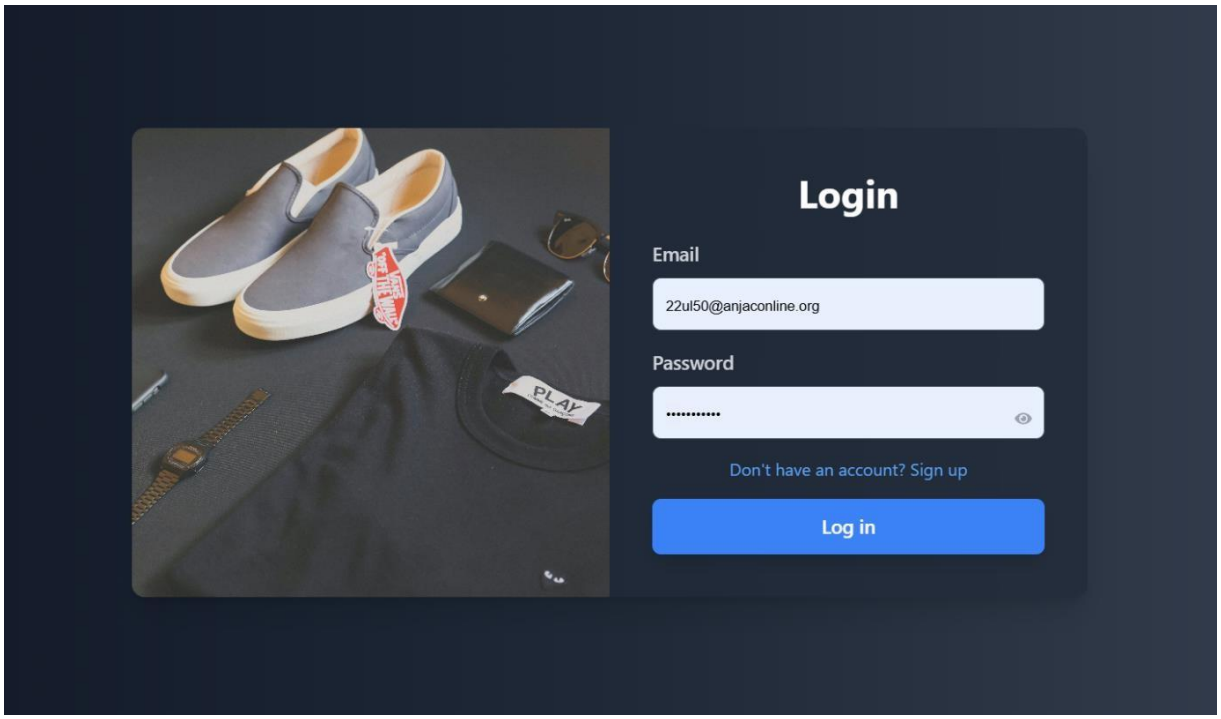


Figure: 5.2.1 User Login

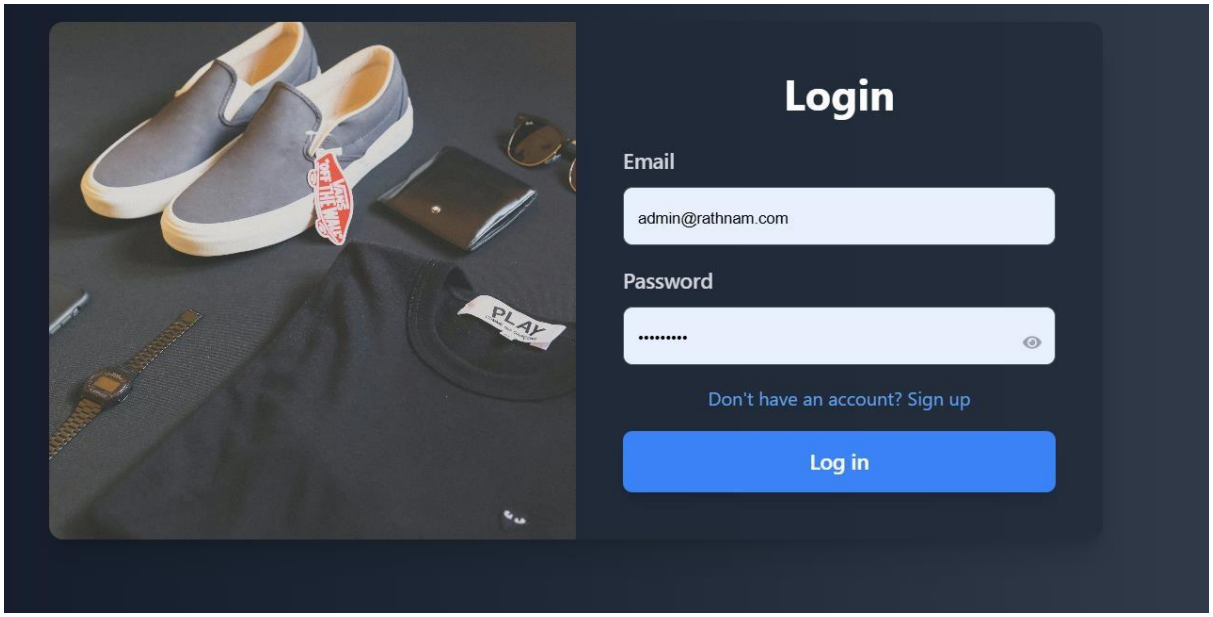


Figure: 5.2.2 Admin Login

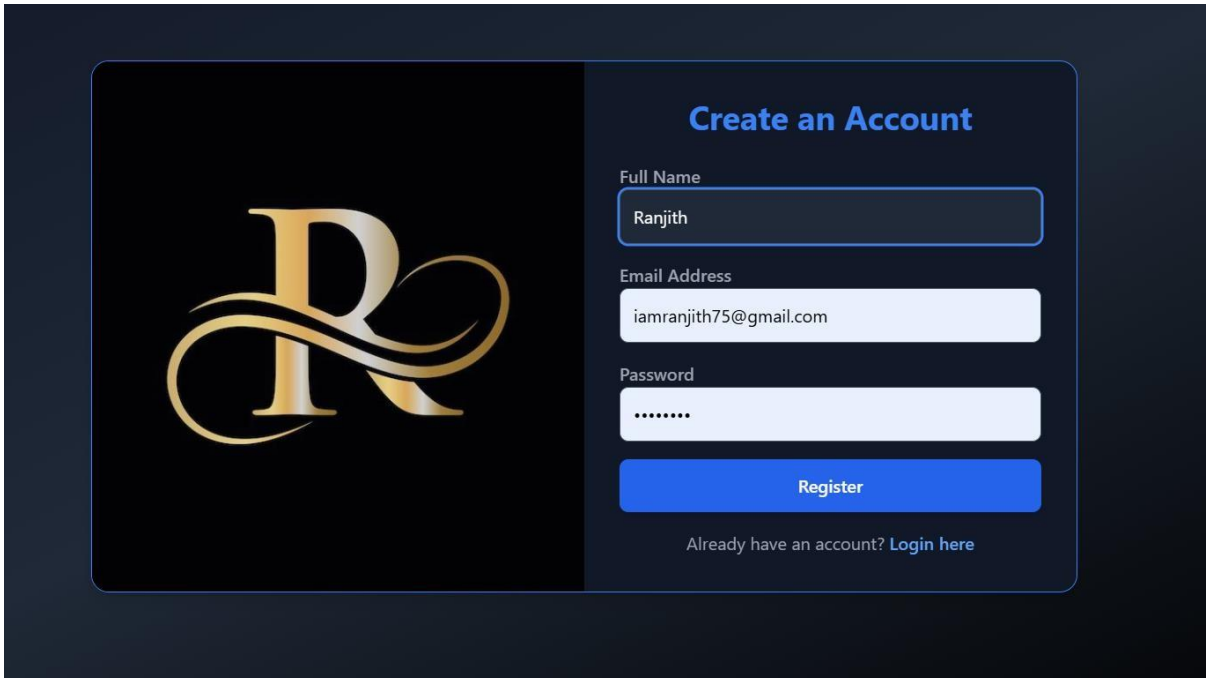
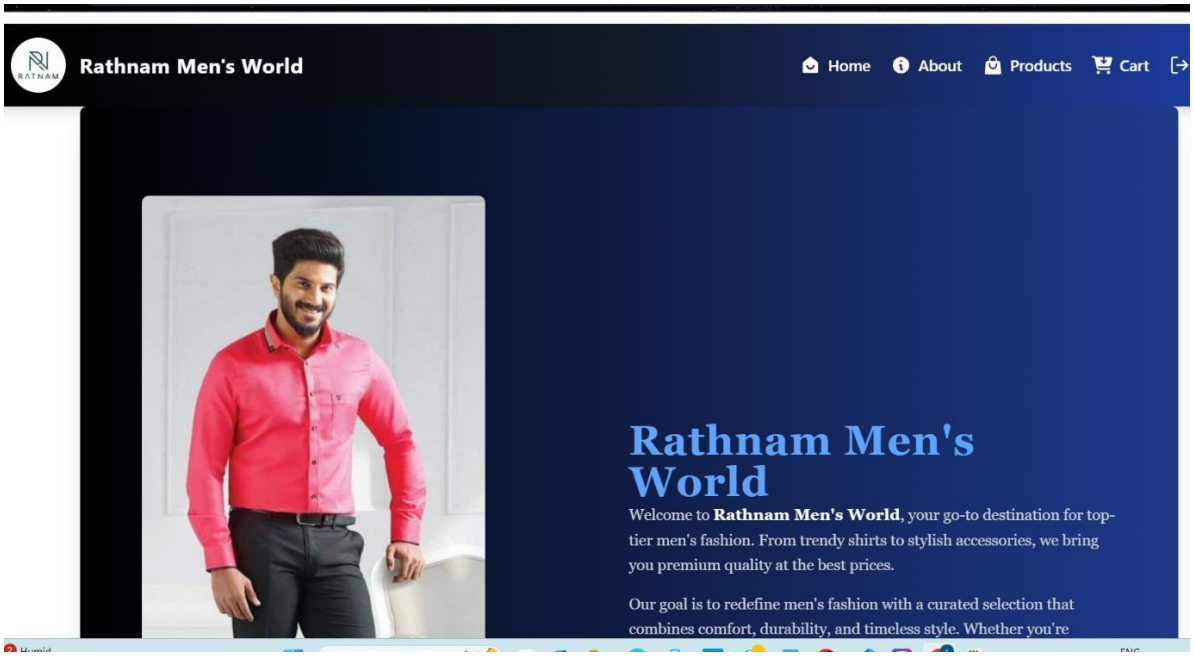


Figure: 5.2.3 User Registration



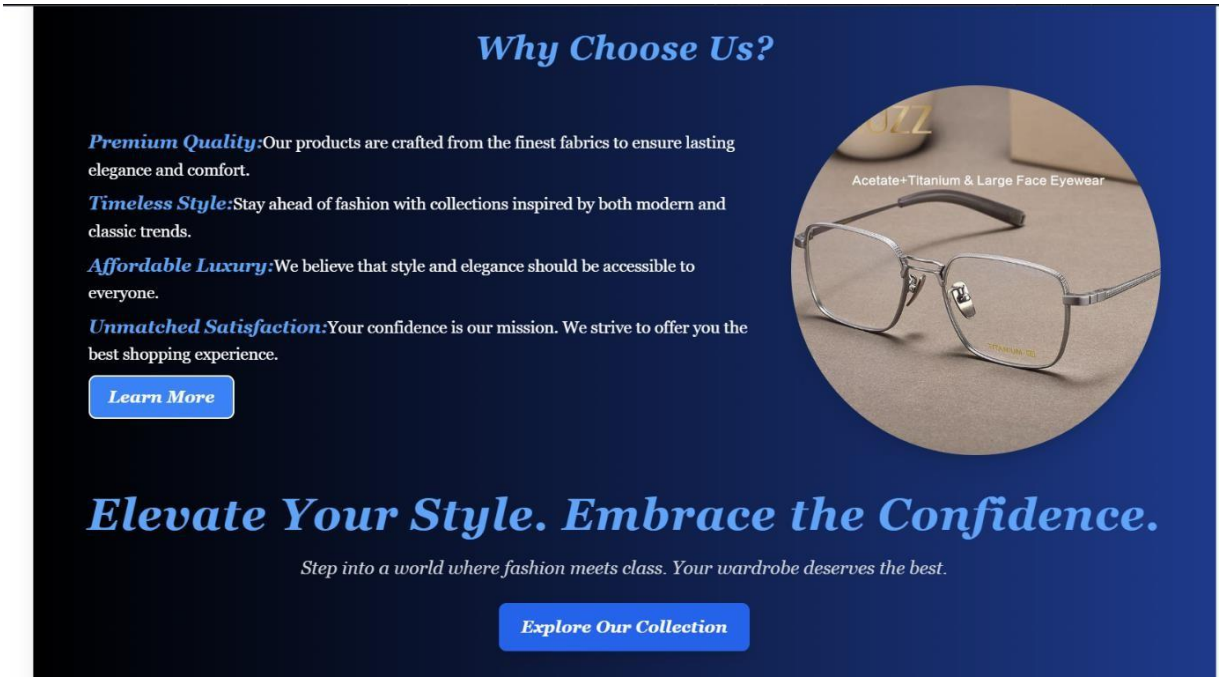


Figure: 5.2.4 Home Page

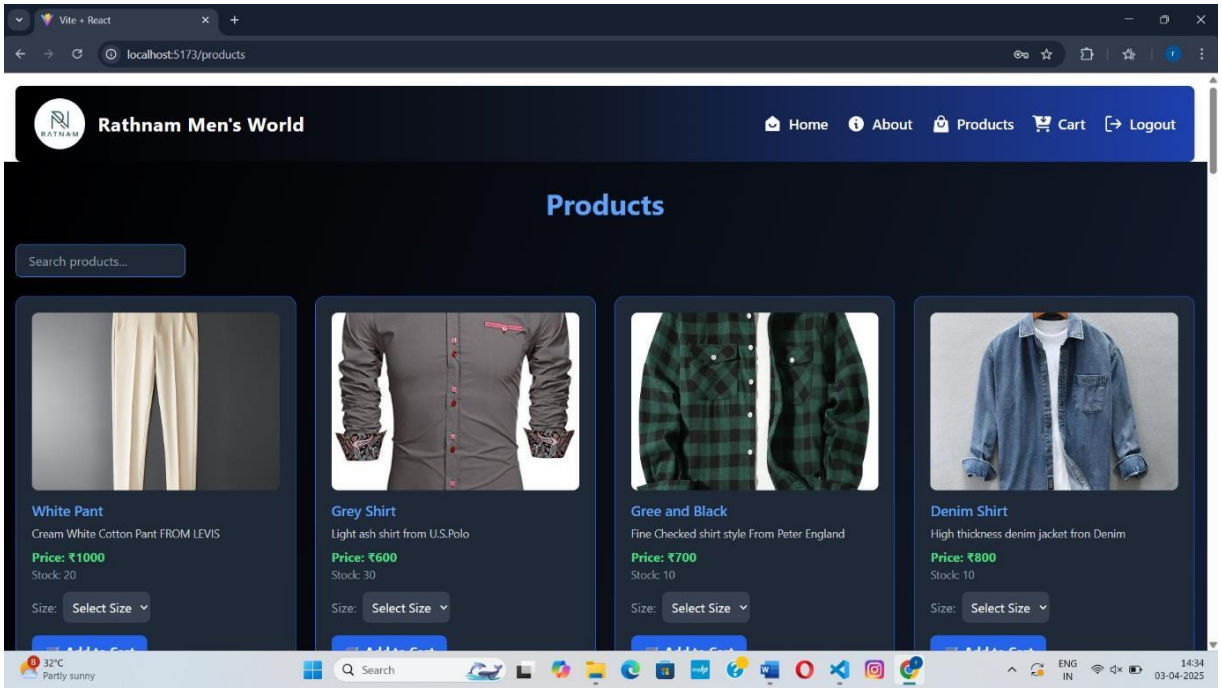


Figure: 5.2.5 Products

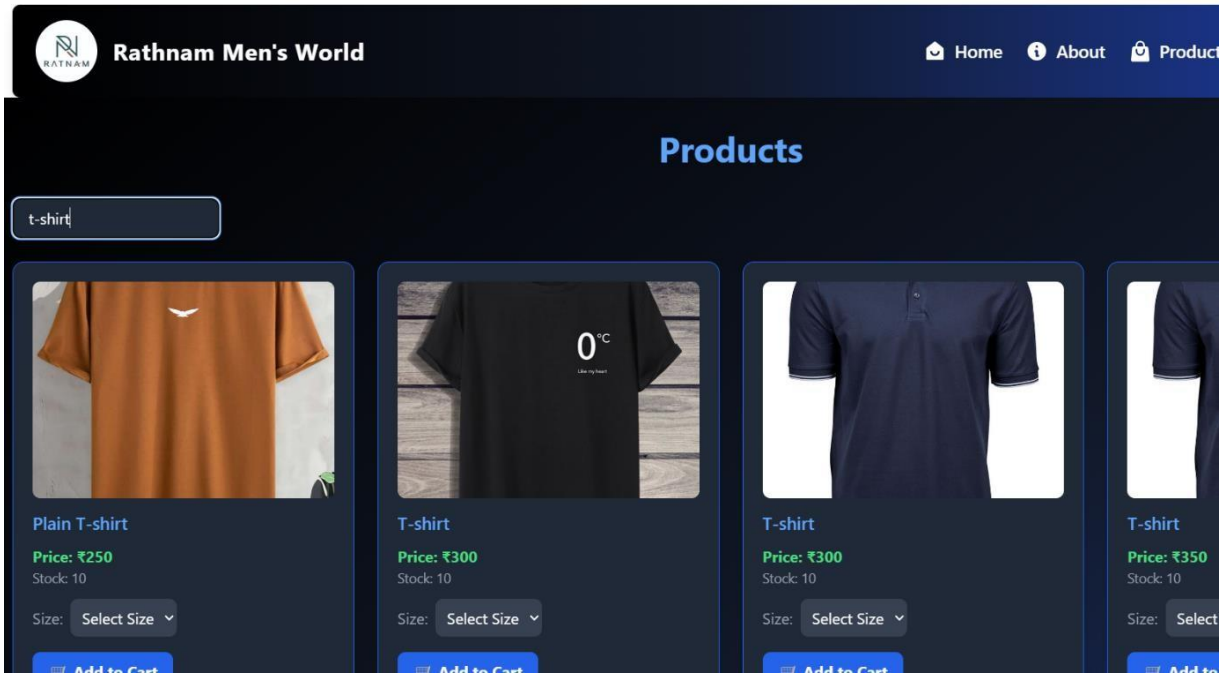


Figure: 5.2.6 T-shirt

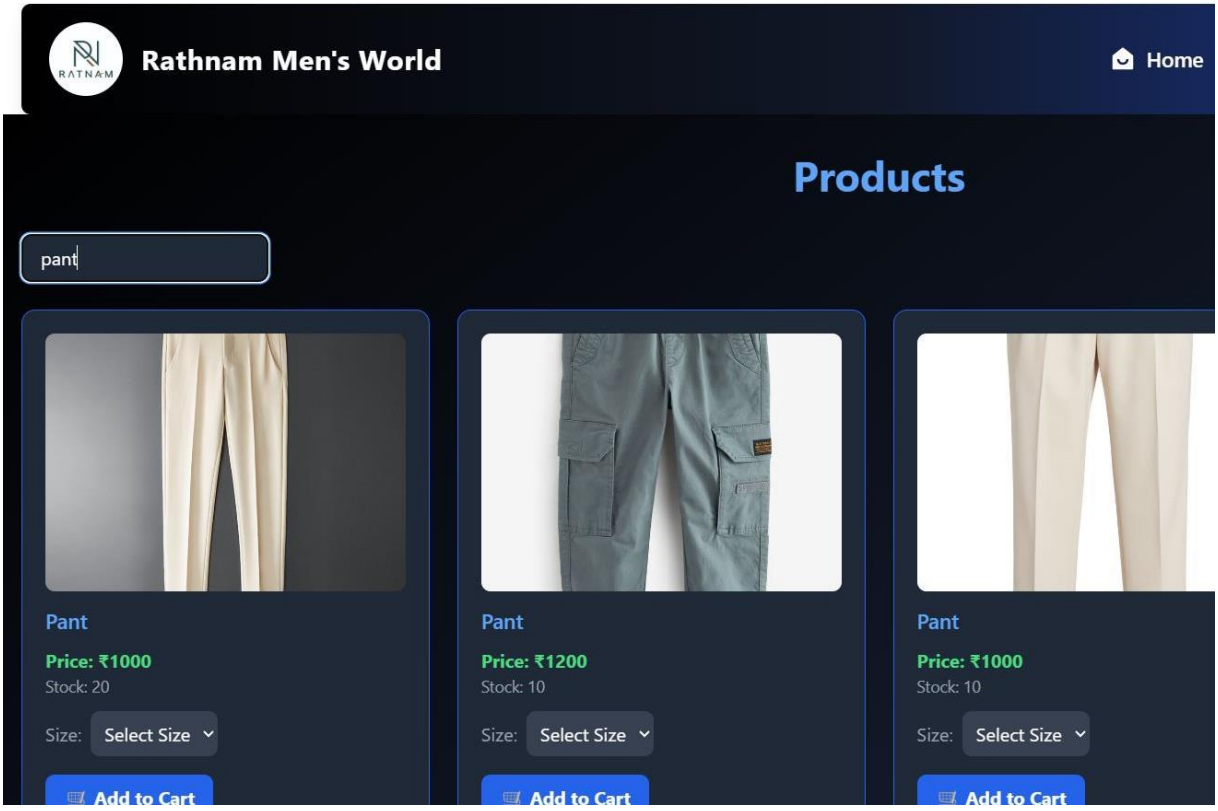


Figure: 5.2.7 Outfit for shirts

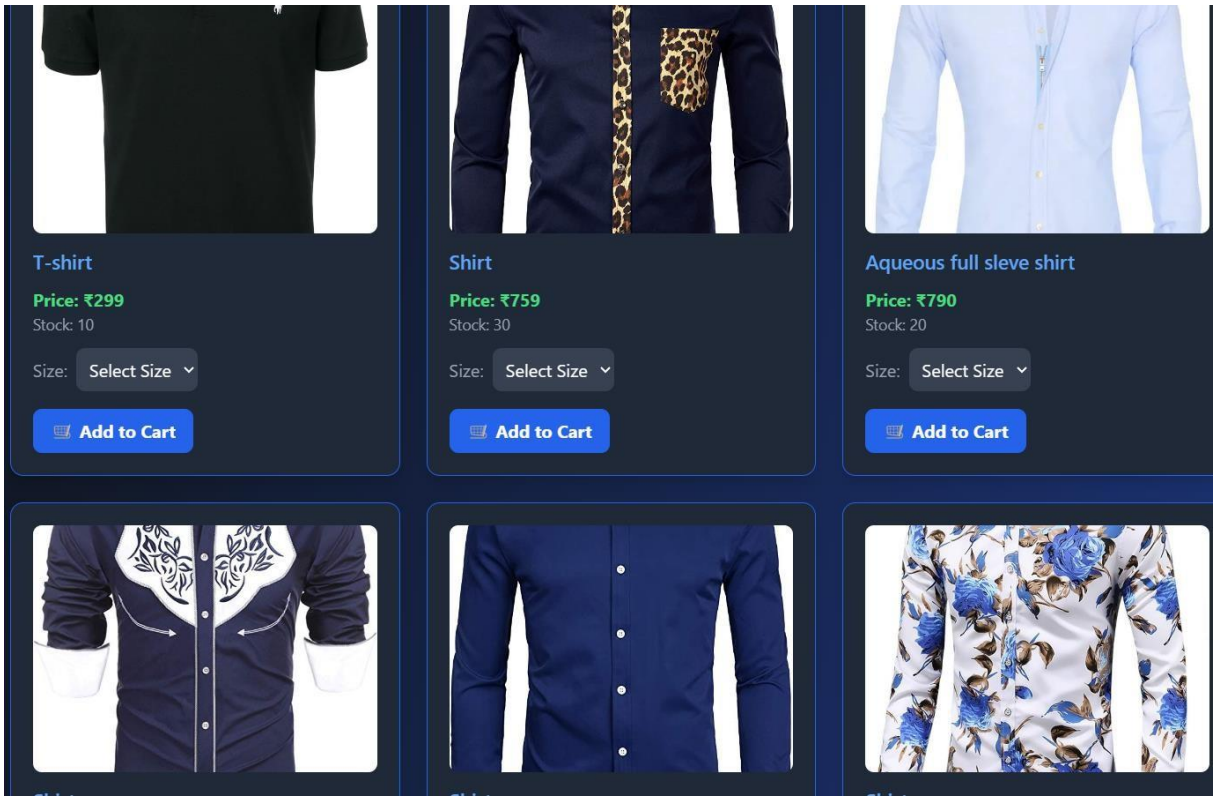


Figure: 5.2.8 Shirts

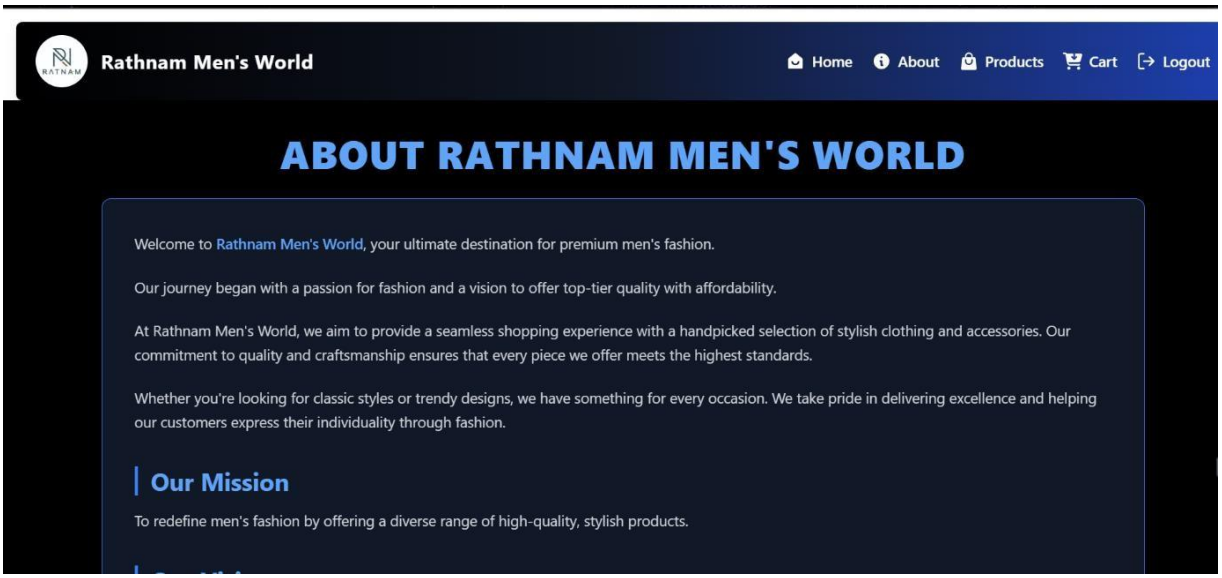


Figure: 5.2.9 About us

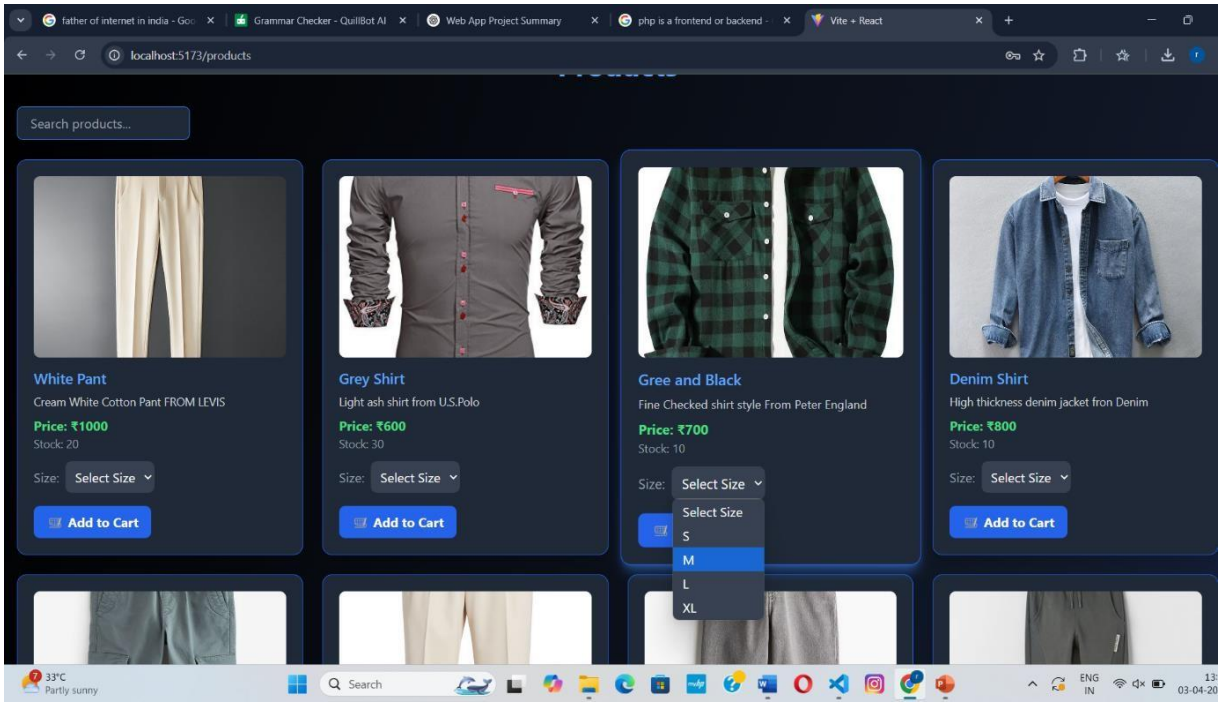


Figure: 5.2.10 Outfit Description

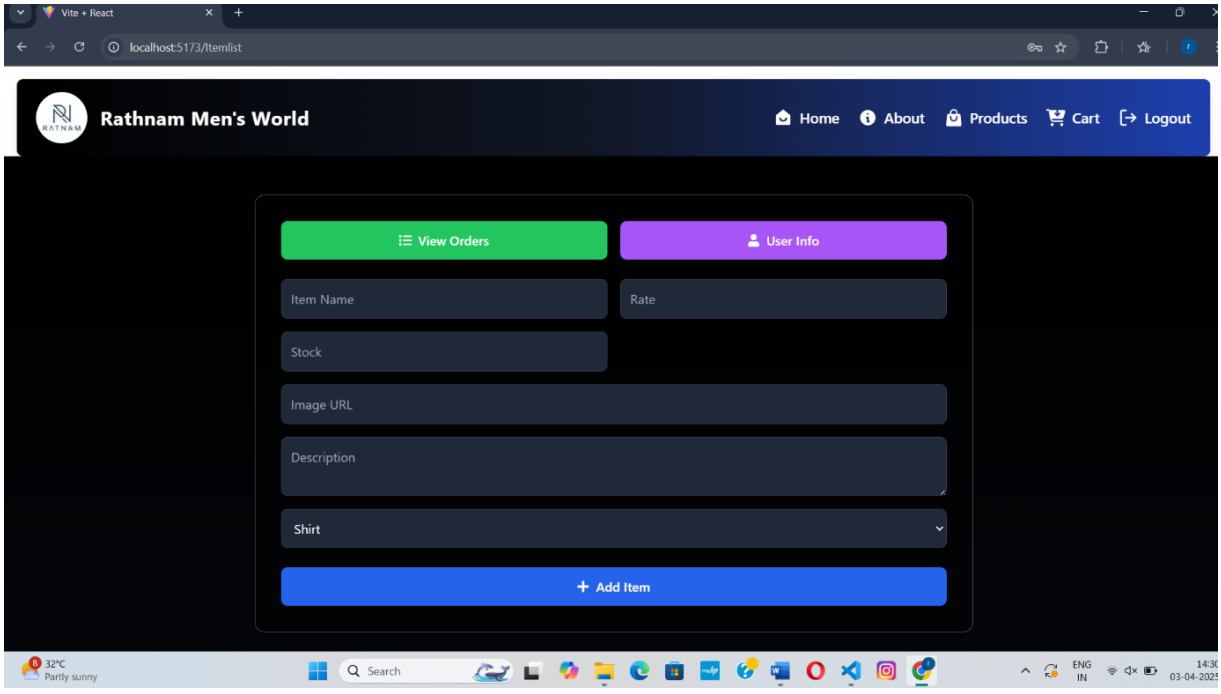


Figure: 5.2.11 Products Update

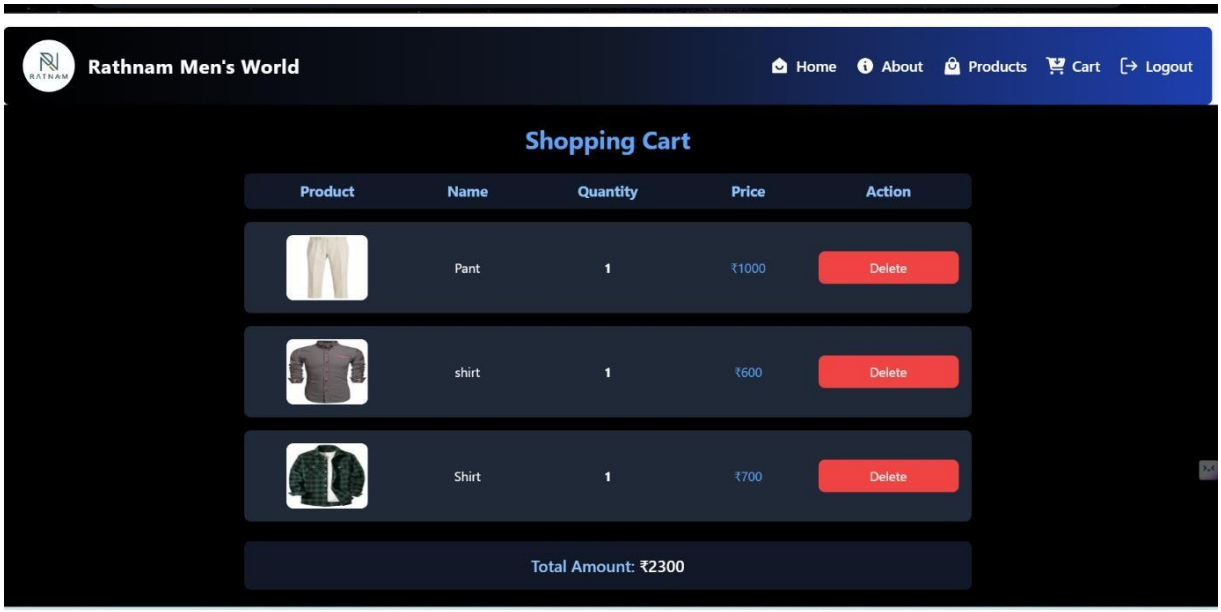


Figure: 5.2.12 Add to Cart

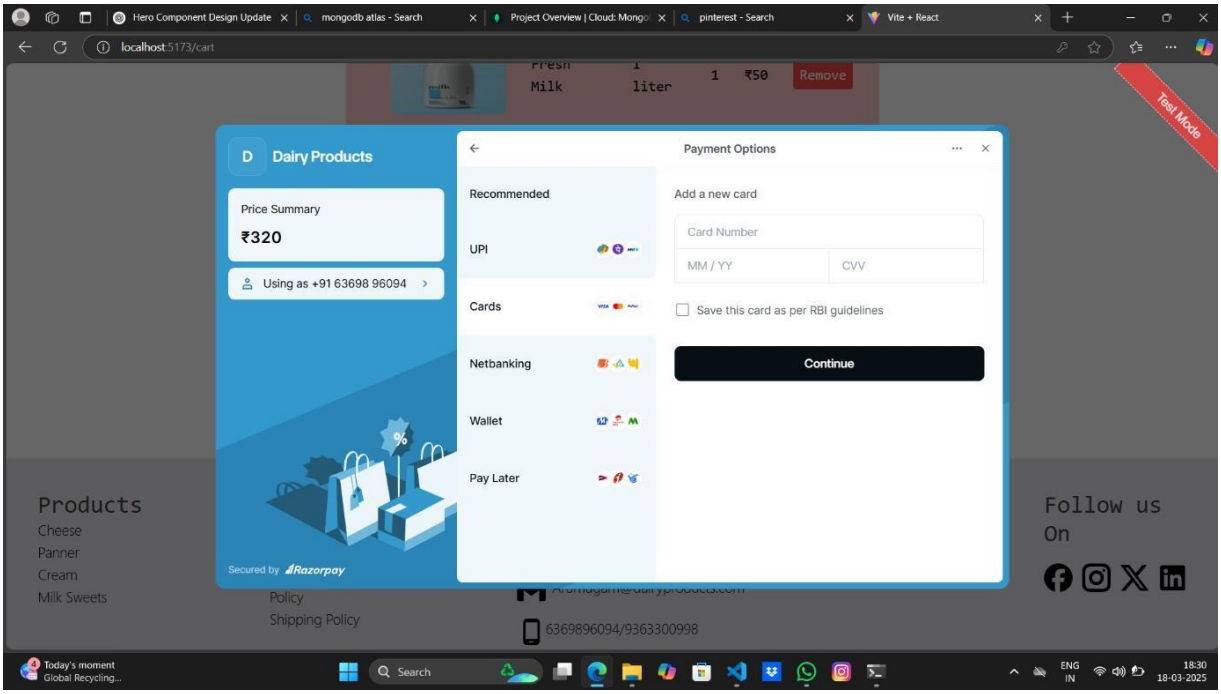


Figure: 5.2.13 Payment methods

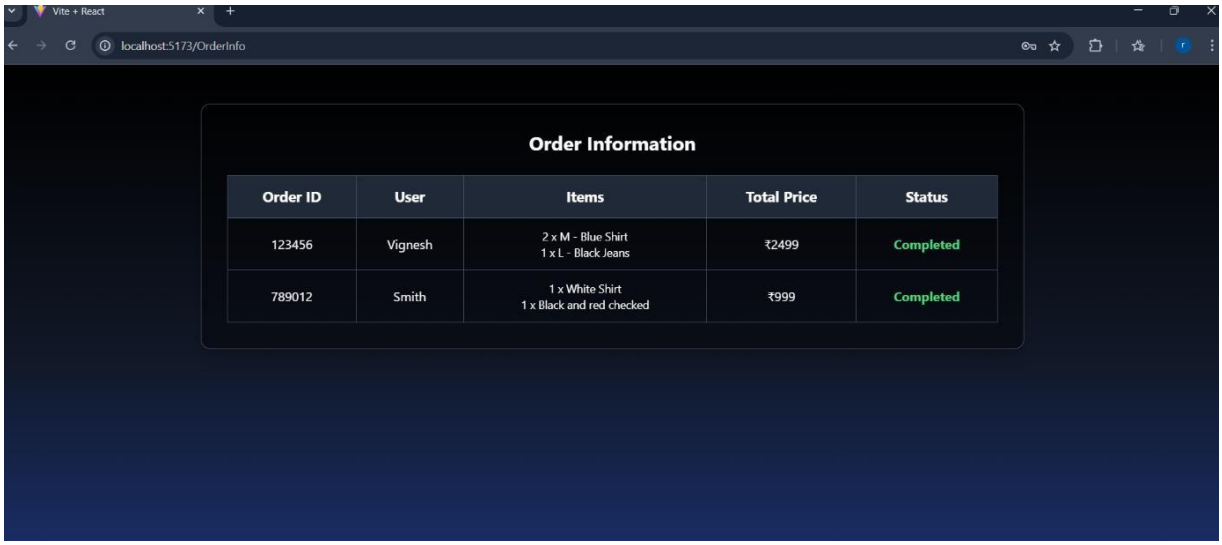


Figure: 5.2.14 Order Info

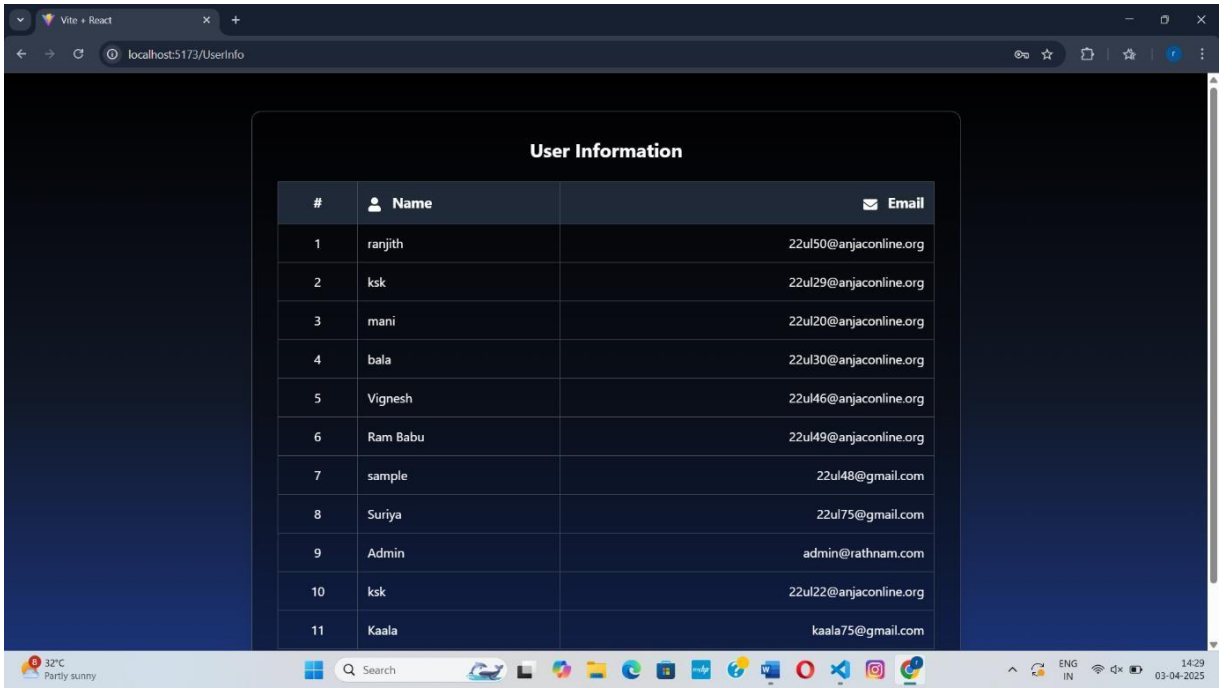


Figure: 5.2.15 User Info

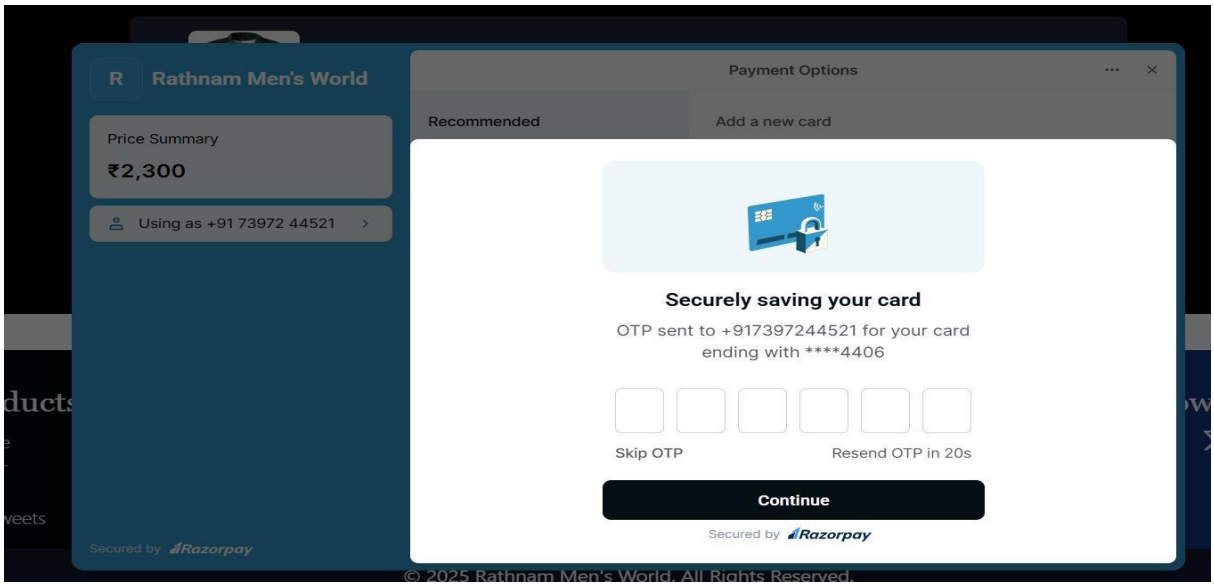


Figure: 5.2.16 Payment

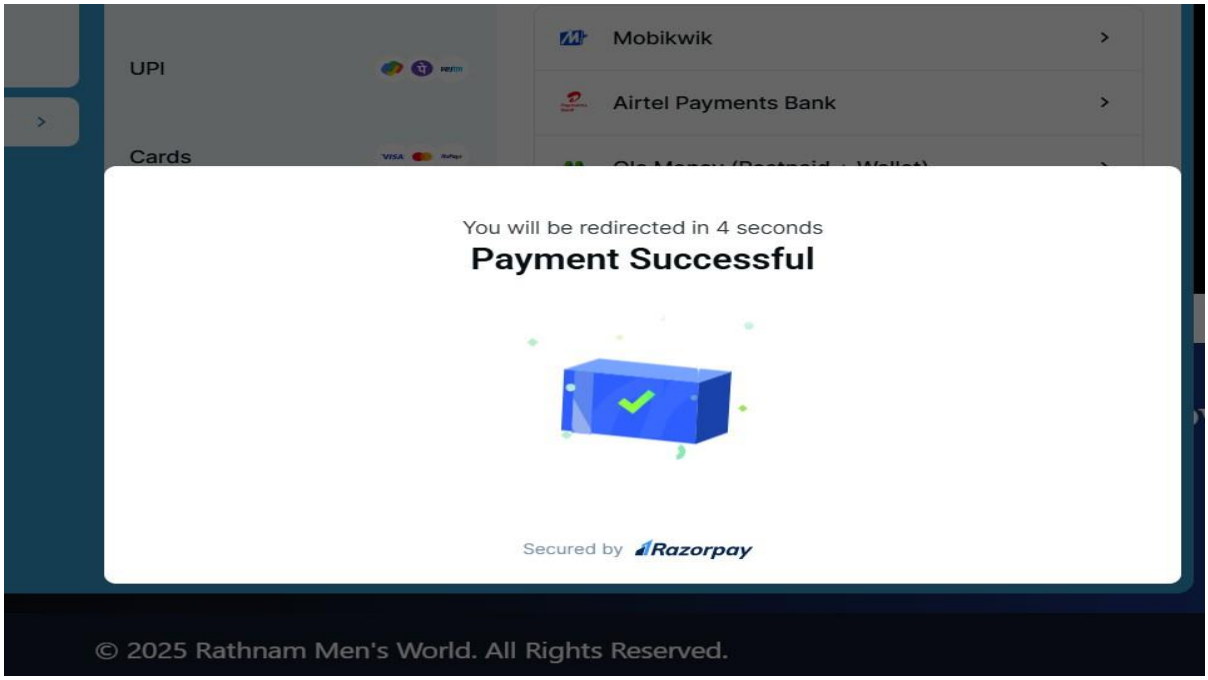


Figure: 5.2.17 Payment success

6. SYSTEM TESTING

System testing of software or hardware is testing conducted on a complete, integrated system to evaluate the system's compliance with its specified requirements. System testing falls within the scope of black box testing, as such, should require no knowledge of the inner design of the code or logic.

As a rule, system testing takes, as its input, all of the "integrated" software components that have passed integration testing and also the software system itself integrated with any applicable hardware systems. The purpose of integration testing is to detect any inconsistencies between the software units that are integrated together or between any of the assemblages and the hardware. System testing is a more limited type of testing; it seeks to detect defects both within the "interassemblages" and also within the system as a whole.

System testing is performed on the entire system in the context of a Functional Requirement Specifications FRS and System Requirement Specification SRS. System testing tests not only the design, but also the behavior and even the believed expectations of the customer. Testing is also intended to test up to and beyond the bounds defined in the software/hardware requirements specifications.

6.1 UNIT TESTING

Firstly, identify the various components or units within your webapp codebase, such as functions, classes, modules, or even smaller pieces like individual methods or API endpoints. After identified these units, create test cases for each one to cover different scenarios and edge cases. These test cases should include inputs that are likely to produce expected outputs as well as inputs that could potentially lead to errors or unexpected behavior. With the test cases prepared, implement the unit tests using a testing framework suitable for our technology stack, such as Jest for JavaScript or Py Test for Python. Next, execute the unit tests to verify that each component behaves as expected in isolation. During execution, pay close attention to any failing tests, which indicate potential bugs or issues that need to be addressed.

Benefits:

- Unit testing increases confidence in changing/ maintaining code. If good unit tests are written and if they are run every time any code is changed, we will be able to promptly catch any defects introduced due to the change. Also, if codes are already made less interdependent to make unit testing possible, the unintended impact of changes to any code is less.
- Codes are more reusable. In order to make unit testing possible, codes need to be modular. This means that codes are easier to reuse.
- The cost of fixing a defect detected during unit testing is lesser in comparison to that of defects detected at higher levels. Compare the cost (time, effort, destruction, humiliation) of a defect detected during acceptance testing or when the software is live.
- Debugging is easy. When a test fails, only the latest changes need to be debugged. With testing at higher levels, changes made over the span of several days/weeks/months need to be scanned.
- Codes are more reliable.
- The goal of unit testing is to segregate each part of the program and test that the individual parts are working correctly. It isolates the smallest piece of testable software from the remainder of the code, and determine whether it behaves exactly as you expect. Unit testing has proven its value in that a large percentage of defects are identified during its use. It allows automation of the testing process, reduces difficulties of discovering errors contained in more complex pieces of the application, and test coverage is often enhanced because attention is given to each unit.
- Unit tests detect changes which may break a design contract. They help with maintaining and changing the code. Unit testing really reduces defects in the newly developed features or reduces bugs when changing the existing functionality. Unit Testing verifies the accuracy of each unit.

Features of Unit Testing

- Software developers write and execute Unit test cases.
- The purpose of unit testing is to isolate each part of the program and make sure each of them working seamlessly.
- It occurs just after coding and debugging and before integration.
- In this project, unit testing was done by every module and each part of the program are executed correctly.
- It is very helpful to run every time when the code is changed.
- It is every to identify the part where we done mistake in program by changing the code.
- Easily we can identify the error that is last we modified in the code.

6.2 INTEGERATION TESTING

Integration Testing is a level of software testing where individual units are combined and tested as a group. The purpose of this level of testing is to expose faults in the interaction between integrated units. Test drivers and test stubs are used to assist in Integration Testing.

- **Component integration testing** -Testing performed to expose defects in the inter faces and interaction between integrated components.
- **System integration testing**-Testing the integration of systems and packages; testing interfaces to external organizations (e.g. Electronic Data Interchange, Internet).
- The Integration test procedure irrespective of the Software testing strategies (discussed above):
- Prepare the Integration Tests Plan
- Design the Test Scenarios, Cases, and Scripts.
- Executing the test Cases followed by reporting the defects.
- Tracking & re-testing the defects.

Steps 3 and 4 are repeated until the completion of Integration is successful.

Integration testing is a crucial phase in the software development process, focusing on verifying the seamless interaction and functionality of individual components when integrated into a larger system. This phase comes after unit testing, where the goal is to ensure that the integrated components work harmoniously together, adhering to defined specifications and requirements. Integration testing involves combining modules, subsystems, or layers of the software architecture and testing their interactions to detect any inconsistencies, interface mismatches, or communication failures.

Various approaches to integration testing exist, including top-down, bottom-up, big bang, and incremental integration. Top-down integration testing begins with testing higher- level modules first, gradually integrating lower-level modules. Conversely, bottom-up integration testing starts with lower-level modules, incrementally integrating higher-level modules. Big bang integration testing involves integrating all components simultaneously, while incremental integration testing proceeds in small increments, adding new functionality and testing the integration at each step.

Regardless of the approach, integration testing plays a vital role in identifying and addressing issues related to interface compatibility, data flow, and communication protocols. By detecting and resolving integration issues early in the development lifecycle, integration testing contributes to the overall reliability, stability, and quality of the software system. It provides assurance that the integrated system behaves as expected, meets user requirements, and is ready for subsequent testing phases and eventual deployment.

Incremental integration testing is an approach in which 2 or more modules with closely related logic and functionality are grouped and tested first, then gradually move on to other groups of modules, instead of testing everything at once. The process ends when all modules have been integrated and tested. Incremental integration testing is more strategic than Big Bang testing, requiring substantial planning beforehand. Incremental integration testing can be further divided into 3 smaller approaches, each also comes with its own advantages and disadvantages that QA teams need to carefully consider for their projects.

Inconsistent code logic

They are coded by different programmers whose logic and approach to development differ from each other, so when integrated, the modules cause functional or usability issues. Integration testing ensures that the code behind these components is aligned, resulting in a working application.

Shifting requirements

Clients change their requirements frequently. Modifying the code of 1 module to adapt to new requirements sometimes means changing its code logic entirely, which affects the entire application. These changes are not always reflected in unit testing, hence the need for integration testing to uncover the missing defects.

Erroneous Data

Data can change when transferred across modules. If not properly formatted when transferring, the data can't be read and processed, resulting in bugs. Integration testing is required to pinpoint where the issue lies for troubleshooting.

Third-party services and API integrations

Since data can change when transferred, API and third-party services may receive false input and generate false responses. Integration testing ensures that these integrations can communicate well with each other.

Inadequate exception handling

Developers usually account for exceptions in their code, but sometimes they can't fully see all of the exception scenarios until the modules are pieced together. Integration testing allows them to recognize those missing exception scenarios and make revisions.

External Hardware Interfaces

Bugs can also arise when there is software-hardware incompatibility, which can easily be found with proper integration testing. When referring to "low-level", we are talking about the very basic building blocks of the software, performing the most fundamental and basic tasks in the system. They are basic data structures or simple functions that perform a specific and low-impact task in the software. On the other hand, when referring to "high-level", we are talking about the most comprehensive and complex components of the system, representing the complete system behaviour.

7. SYSTEM MAINTENANCE

System maintenance plays a crucial role in ensuring the smooth, secure, and uninterrupted operation of the eCommerce platform. It includes regular updates, performance monitoring, data integrity checks, and security auditing to maintain the health of both frontend and backend components of the application.

Maintenance Activities

Maintenance activities in an eCommerce project are essential to ensure the application remains functional, secure, efficient, and user-friendly over time. These activities help identify and fix issues, upgrade systems, and adapt to user and business needs.

Bug Fixing

Resolving issues found during testing or reported by users, such as broken UI components, failed payments, or incorrect product data.

Performance Optimization

Improving website speed, optimizing database queries, compressing images, and minimizing response time for a smoother user experience.

Security Updates

Applying patches, updating dependencies, and securing sensitive data (like customer info and payments) to protect against cyber threats.

Backup and Recovery

Regularly backing up the database and user data to avoid data loss due to accidental deletion, server crashes, or attacks.

System Updates

Updating frameworks, libraries (like React, Node.js), and dependencies to their latest stable versions for better performance and compatibility.

Monitoring and Logging

Using tools to monitor uptime, API health, and user behavior, and analyzing logs to detect anomalies or failures.

Database Maintenance

Removing unnecessary data, indexing collections, and ensuring data integrity in MongoDB to maintain performance.

Importance

IT improves security, efficiency, productivity and helps make business processes run smoothly but IT systems require regular maintenance to ensure the systems themselves continue to work efficiently.

Most of the repetitive maintenance activities carried out on IT systems are done manually but can be easily automated. Manual maintenance activities can often be poorly completed or may not be completed at all as technicians forget to perform maintenance activities on schedule. This can lead to eventual systems failure and operational downtime, both of which can be very expensive. Automation of IT services can prevent this unnecessary expenditure and also reduce periods of non-productivity for staff.

Automation also allows for on demand and scheduled reporting which provides the ability to understand your IT systems health status anytime. This also generates alerts when maintenance activities fail so that they can be manually completed by an administrator.

In this project, maintenance is focused on upgrading an application to ensure it remains productive and cost effective**.** The maintenance is the modification of an application to correct faults; to improve performance; or to adapt the application to a changed environment or changed requirements. Thus, adding new functionality to an existing application (i.e., enhancement) considered maintenance.

8. CONCLUSION

The system is entitled as **Web app for Rathnam Men's World** has been successfully developed using React, Express, Node.js, and MongoDB as the technology stack. The backend is powered by MongoDB Server, ensuring efficient data management. This system is designed to help the administrator maintain detailed records of customers and products. In a world where online shopping is an essential part of daily life, Rathnam Men's World provides a seamless and intuitive platform for purchasing men's clothing such as shirt, t-shirt and pants. With its user-friendly interface, outfits categories, and commitment to customer satisfaction, this platform aims to redefine the online shopping experience for men's fashion, making it easier for customers to explore and purchase high-quality outfits.

BIBLIOGRAPHY

Referred books

[A 21] Adam D. Scott., "Full Stack Development with MongoDB and Express", First Edition, O'Reilly Media, USA, 2021

[A 22] Alex Banks, Eve Porcello., "Learning React", Third Edition, O'Reilly Media, USA, 2022

[E 20] Ethan Brown., "Web Development with Node and Express", Second Edition, O'Reilly Media, USA, 2020

[R 21] Robin Wieruch., "The Road to React", 2021 Self-Published, Germany, 2021

[K 22] Kirupa Chinnathambi., "JavaScript Absolute Beginner's Guide", Second Edition, Pearson Education, USA, 2022

Referred websites

• https://www.tutorialsteacher.com/mern

• https://www.mongodb.com/docs/manual/tutorial/

• https://www.w3schools.com/react/

• https://www.w3schools.com/nodejs/

• https://www.w3schools.com/express/